ChatDB: Obligation-Driven Proof Search with Adversarial Completeness Checking

Design Specification v2.2

Obligation-Driven Search · Adversarial Checking · Honest Scope Claims

Vario Automation · February 26, 2026 · Internal Architecture Document

---

Executive Summary

On February 26, 2026, ChatDB completed its first full proof run on IMO 2025 Problem 3 (bonza functions). The result: 19 steps verified, 0 rejected,

100% local accuracy, wrong answer. The system proved c = 3 when the correct answer is c = 4. Every algebraic step was mechanically validated. The failure was not in verification but in exploration.

This specification defines the architecture that closes this gap. It introduces obligation-driven proof search over a directed acyclic graph (DAG) with adversarial completeness checking. The key insight: verification confirms steps, obligations confirm coverage. Without obligations, a proof engine converges on the first locally-consistent path regardless of whether it leads to the correct answer.

Core Thesis: Alignment by control flow, not alignment by prompt. The model can hallucinate freely — the obligation table doesn't care. `SELECT COUNT(*) FROM obligations WHERE status = 'open'` returns a number, and if that number isn't zero, the loop doesn't terminate. No amount of "Therefore, QED" in the output changes that.

What This System Is: ChatDB is an architecture for *institutionalized mathematical reasoning*. It works by encoding human mathematical practice — technique families, construction ontologies, case analysis patterns — into enforceable runtime structures. Its effectiveness scales with the density of available technique ontologies.

What This System Is Not: ChatDB does not provide general epistemic completeness. It cannot guarantee discovery of unknown unknowns. It makes ignorance legible, persistent, and expensive to ignore. That is a strong property — strong enough to beat competition mathematics — but it is not the same as completeness.

Changes from v2.1:

- Honest scope claims throughout (what the system can and cannot guarantee)
- Obligation lifecycle expanded: `superseded` and `retracted` states for pruning
- Obligation explosion prevention: fan-out limits, global ceiling, mass invalidation
- Layer 4 reframed as growing pattern library, not mechanical oracle
- Adversary confidence modulated by coverage, not just budget expenditure
- OODA self-modification gets in-run regression testing
- Concurrency model specified (event-sourced DAG)
- Context window management for agent queries
- Cold start / bootstrapping protocol
- Completion quality metric (closed vs. demoted distinction)
- Realistic 6-month timeline

Part 1: The Evidence

IMO 2025 Problem 3

Problem Statement: Let N be the set of positive integers. A function f: N → N is called bonza if f(a) divides b^a − f(b)^f(a) for all positive integers a and b. Determine the smallest real constant c such that f(n) ≤ c·n for all bonza functions f and all positive integers n.

Known answer: c = 4. System's answer: c = 3.

Run Metrics:

Metric	Value	Significance
Total steps	19	Full proof chain produced
Verified	19	Every step passed SymPy validation
Rejected	0	No algebraic errors detected
Final answer	c = 3	Incorrect — correct answer is c = 4
Tokens	44.7k in / 8.1k out	Single-model linear run

Diagnosis: The solver explored power functions f(p) = p^k at prime inputs exhaustively. It correctly established f(1) = 1, f(a) | a^a, and that the maximum ratio for odd primes via power functions is 3 (achieved at f(3) = 9). It then correctly proved that f(p) = p^(p−1) is unachievable for p ≥ 5 due to constraints from b = 2. Every algebraic step in this chain is correct.

The solver never considered exponential constructions involving prime factorization, multiplicative functions, or piecewise constructions where f(n) = 2^e(n) type functions achieve ratio 4. The search space was explored narrowly but deeply. The technique class "power functions at primes" was exhausted; "exponential/multiplicative constructions" was never entered.

Step 19 — Death Spiral: The model argued with itself in the final step, alternating between "c = 3/2" (memorized prior) and "c = 3" (consistent with its verified chain). It produced: "c = 3/2… wait… NO… FINAL: c = 3/2 is incorrect. The answer is c = 3." This death spiral is structured signal that the model's prior conflicts with its evidence.

Root Cause Analysis:

1. No exploration tracking. The engine cannot distinguish "I proved this path doesn't work" from "I never looked at this path."
2. No obligation enforcement. When the solver establishes f(n) | n^n, this opens a case analysis across construction families. The current engine treats it as one step and moves on.
3. No adversary. No agent tries to break the solver's claimed bound. An adversary would have found f(n) = 2^(v_2(n)) and forced the solver to account for c = 4.
4. No obligation parsing. Even if the engine tracked obligations, it has no mechanism to detect that "f(n) | n^n" is an obligation generator. The

step looks like a lemma. It's actually a branching explosion disguised as a lemma.

Design Caveat: This spec is *fitted* to the P3 failure. Every feature addresses something that went wrong on P3. Before v2.2 ships, it must be validated blind against 5-10 problems from different mathematical domains where the failure mode was not anticipated during design. P3 is the training set. We need a test set.

Part 2: Scope and Honest Limitations

This section exists because the strongest version of this spec requires honest framing.

What Obligation Tracking Actually Provides

Obligation tracking turns exploration into something *auditable*, not something *objective*. The obligation table is a completeness *pressure system*. It forces the solver to address known unknowns. It cannot force the solver to discover unknown unknowns.

Specifically:

The system CAN guarantee: No proof terminates with known-open search spaces unaddressed. Every detected obligation is either resolved, escalated through multiple attempts, or explicitly demoted with the confidence cost recorded. The system maintains a legible record of what it explored and what it didn't.

The system CANNOT guarantee: That all obligations were detected. That the technique ontology covers the problem's actual solution space. That the adversary's search was adequate. That demotion didn't discard the critical path.

The gap is honest: Unknown unknowns — construction families not in Layer 4's pattern library, techniques not in any classifier's training data, approaches not in the technique registry — are invisible to the obligation system. They are addressed indirectly through arxiv search, librarian coverage monitoring, and adversary speculation, all of which are heuristic.

Resource-Bounded Epistemic Closure

The completeness invariant (`while open_obligations > 0 && budget > 0`) provides *resource-bounded epistemic closure*, not mathematical completeness. Different token budgets produce different "complete" proofs. This is by design — the alternative (unbounded search) is not viable. But we name it honestly.

Every proof conclusion carries a **completion quality metric** (Section 10) that distinguishes "all obligations genuinely resolved" from "remaining

obligations demoted by budget exhaustion." These are categorically different confidence levels.

Domain Scope

ChatDB's effectiveness correlates with the density of institutionalized mathematical practice in the problem domain.

Strong coverage: Competition mathematics, algebraic number theory, functional equations, elementary combinatorics. These domains have well-catalogued technique families and construction ontologies.

Moderate coverage: Analysis, geometry, probability. Technique families exist but are less discrete; the boundary between "explored" and "unexplored" is fuzzier.

Weak coverage: Novel interdisciplinary problems, research-frontier questions, domains where the relevant techniques don't have names yet. Here, Layer 4 contributes little, Layer 3 is unreliable, and the system degrades to Layer 1 (self-tagging) quality.

The architecture (DAG, obligation table, multi-model orchestra, technique registry) is domain-general. The obligation parsing pipeline is domain-specific and its coverage grows over time as the pattern library expands.

Part 3: Pre-Solve Intelligence Gathering

Before the solver generates a single step, four agents work in parallel to build an intelligence briefing. The solver walks in informed, not blind.

Agents

Problem Analyst — Classifies problem type, identifies domain, extracts key mathematical structures. Output: structured problem profile.

Technique Scout — Queries technique registry for known approaches, then searches arxiv via MCP for recent methods. Ranks technique families by historical success rate. For P3: finds "exponential constructions" and "p-adic valuation methods" alongside the obvious "substitution strategies." Output: ranked technique list with success rates.

Constraint Pre-Computer — Runs SymPy on the problem statement itself. For P3: computes divisors of n^n for n = 2,3,4,5,6. Identifies function families satisfying f(a)|b^a − f(b)^f(a) for small a,b. Pre-enumerates the search space that Layer 4 would discover later — but does it BEFORE the solver starts. Output: pre-computed constraint landscape and initial obligation candidates.

Similar Problem Retriever — Searches problem database and arxiv for structurally similar problems and their known solution techniques. Output: analogies, warnings, and solution patterns.

Integration

All four agents run in parallel. Total latency: ~2 seconds. Total cost: ~$0.50-1.00 (four Haiku-class calls). Their outputs merge into a pre-solve briefing that is:

1. Injected into the solver's initial prompt as structured context
2. Used by the orchestrator to seed initial obligation priorities
3. Used by the librarian as its baseline coverage expectation
4. Used by the adversary to begin speculative counterexample search immediately

The pre-solve briefing doesn't constrain the solver — it informs it. The solver can ignore the briefing and explore novel directions. But it can't claim ignorance of known technique families.

```
CREATE TABLE pre_solve_briefings (
  id TEXT PRIMARY KEY,
  problem_id TEXT NOT NULL REFERENCES problems(id),
  problem_profile TEXT NOT NULL,
  technique_ranking TEXT NOT NULL,
  pre_computed_constraints TEXT,
  similar_problems TEXT,
  initial_obligations TEXT,
  total_cost REAL,
  created_at TEXT NOT NULL
);
```

Part 4: The Obligation Parsing Pipeline

The Hard Problem

Obligations are only useful if the engine reliably detects when a proof step opens a search space. The statement "f(n) | n^n" is a constraint that implies a case analysis across all divisor families of n^n. But it doesn't announce itself as such. No trigger phrase. No syntactic marker. The search debt is semantic, not syntactic.

We address this with a three-layer hybrid pipeline. Each layer has a different failure mode. Their union catches what any single layer misses.

Design Principle: Push intelligence into schema, not prompts.

Layer 1: Model Self-Tagging

The solver's prompt includes instructions to annotate each step with obligation metadata as a JSON sidecar.

```
        {
          "step_type": "claim | case_split | reduction | bound |
construction",
          "technique_class": "substitution | modular_arithmetic | ...",
          "construction_family": "power_at_primes | exponential | ...",
          "opens_search_space": true,
          "search_branches": [
            { "description": "...", "obligation_type": "case |
construction | bound" }
          ]
        }
```

Catches: Explicit case splits, stated reductions, announced constructions.

Misses: Implicit search spaces, unknown unknowns, motivated reasoning (model undercounts obligations to reach conclusion faster).

Reliability: Low for the obligations that matter most. ~60% of syntactically obvious case splits, ~10% of semantically implicit search spaces. Its primary value: it's free (part of the solver's existing output) and it's right when it fires (~95% precision on explicit case splits).

Trust Level: Layer 1 is a hint channel, not a source of truth. Layer 1 is *never* trusted to assert absence of obligations.

Layer 3: Obligation Classifier Ensemble

A group of 2-3 separate models (distinct from the solver) reads each verified step in the context of the full chain and problem statement. Their sole job: detect whether the step opens a search space and enumerate the branches.

Ensemble composition: Different model families to provide diverse blind spots. Example: Claude Haiku + GPT-4o-mini + DeepSeek. All run in parallel on every verified step.

Merge rule: UNION of detected obligations (not intersection), with a precision governor (see Section 5: Obligation Management).

```
You are an obligation detector for mathematical proof search.
Your job: identify when a proof step opens unexplored search
space.

PROBLEM: {problem_statement}
DOMAIN: {domain}
CURRENT CHAIN: {verified_steps_summary}
NEW VERIFIED STEP: {step_content}

Does this step open a search space that the proof must explore
to be COMPLETE?

Respond with JSON:
{
```

```
  "opens_obligations": true | false,
  "reasoning": "...",
  "obligations": [
    {
      "description": "specific thing to explore",
      "obligation_type": "case|construction|bound|adversarial",
      "priority": 0.0-1.0,
      "why_missing_matters": "what goes wrong if this is
skipped",
      "confidence": 0.0-1.0
    }
  ],
  "technique_families_referenced": [...],
  "technique_families_absent": [...]
}
```

Catches: Implicit search spaces, non-obvious reductions, cross-step obligations.

Misses: Computational enumeration (can't enumerate all divisors of 12^12). Domain-specific blind spots if the concept is poorly represented across all model families.

Critical Limitation: Model diversity helps with *representational* blind spots (different training emphasis) but not *knowledge* blind spots (if no model in the ensemble has strong signal on p-adic methods, the union of three zeros is still zero). The arxiv search in the pre-solve phase and the librarian's continuous search partially mitigate this, but it is an irreducible limitation.

Cost: ~$0.003 per step for all three classifiers. For a 19-step proof: ~$0.06 total.

The classifier is not the librarian. The classifier fires on every verified step and asks "does THIS step open obligations?" The librarian monitors the ENTIRE run and asks "what has the run missed?" Different triggers, different prompts, different outputs.

Layer 4: Validator-Generated Obligations (Growing Pattern Library)

v2.2 Reframe: Layer 4 is a *growing library of domain-specific obligation extractors*, not a mechanical oracle. It is deterministic and model-independent for the patterns it recognizes. It is silent on patterns it doesn't recognize. The spec's confidence in Layer 4 is scoped to its current pattern library, not to the mathematical universe.

After SymPy verifies a step, an extended validation pass analyzes the step's mathematical content and computes any search spaces it opens using recognized patterns.

```
        def extract_obligations(step_content: str, sympy_result: dict) -
> list[Obligation]:
```

```
            obligations = []

            # Pattern 1: Divisibility constraint f(n) | g(n)
            if is_divisibility_constraint(step_content):
                dividend = extract_dividend(step_content)
                for n in representative_values(dividend):
                    divs = sympy.divisors(evaluate(dividend, n))
                    families = classify_divisor_families(divs, n)
                    for family in families:
                        obligations.append(Obligation(
                            type='construction',
                            description=f'Explore {family.name}
constructions for f({n})',
                            search_space=family.members,
                            priority=family.max_ratio / n,
                            source='validator_divisor_enumeration'
                        ))

            # Pattern 2: Bound claim max f(n)/n = c
            if is_bound_claim(step_content):
                bound_value = extract_bound(step_content)
                obligations.append(Obligation(
                    type='adversarial',
                    description=f'Find bonza function with ratio >
{bound_value}',
                    priority=0.9,
                    source='validator_bound_detection'
                ))

            # Pattern 3: Modular constraint
            if is_modular_constraint(step_content):
                modulus = extract_modulus(step_content)
                survivors = compute_residue_survivors(modulus,
step_content)
                if len(survivors) > 1:
                    for s in survivors:
                        obligations.append(Obligation(
                            type='case',
                            description=f'Check value {s} (mod
{modulus})',
                            priority=0.5,
                            source='validator_residue_enumeration'
                        ))

            # Pattern 4: Function space characterization
            if is_function_constraint(step_content):
                families = enumerate_function_families(step_content)
                for fam in families:
```

```
                    obligations.append(Obligation(
                        type='construction',
                        description=f'Explore {fam} function family',
                        priority=0.6,
                        source='validator_function_enumeration'
                    ))

            # META-PATTERN: Unknown pattern detection
            # If the step contains mathematical structure that doesn't
match
            # any recognized pattern, flag it for Layer 3 escalation
            if has_unrecognized_structure(step_content, sympy_result):
                obligations.append(Obligation(
                    type='review',
                    description='Step contains unrecognized mathematical
structure',
                    priority=0.3,
                    source='validator_unknown_pattern'
                ))

            return obligations
```

What Layer 4 catches that models miss: On the P3 run, Step 9 establishes f(n) | n^n. SymPy computes: for n = 4, divisors of 256 include {1, 2, 4, 8, 16, 32, 64, 128, 256}. These can be generated by power functions AND by exponential functions. The validator detects multiple generating families and opens obligations for each. This is computation, not reasoning.

What Layer 4 cannot do: Strategic obligations, WLOG detection, proof strategy. These are exactly what Layer 3 catches. Additionally, Layer 4 cannot detect construction families not in its `classify_divisor_families` library. The library grows as new patterns are discovered and added (see "Pattern Library Extension" below).

Layer 4 confidence is conditional: Layer 4's 0.95 confidence applies to obligations it *generates*. It says nothing about obligations it *fails to generate*. Layer 4 silence is not evidence of absence — it is evidence that the step didn't match any recognized pattern. This distinction is critical and was underspecified in v2.1.

Pattern Library Extension

Layer 4's pattern library is a registry, not a hardcoded list. New patterns are added without code changes to the core extractor:

```
        CREATE TABLE obligation_patterns (
          id TEXT PRIMARY KEY,
          pattern_name TEXT NOT NULL,
          domain TEXT NOT NULL,
```

```
            detection_function TEXT NOT NULL,  -- Python function name
            description TEXT NOT NULL,
            example_step TEXT,
            example_obligations TEXT,          -- JSON
            added_by TEXT DEFAULT 'manual',    -- 'manual', 'council',
'meta_agent'
            success_count INTEGER DEFAULT 0,
            false_positive_count INTEGER DEFAULT 0,
            created_at TEXT NOT NULL
        );
```

When the council review identifies an obligation that Layer 4 should have caught but didn't (because the pattern wasn't in the library), a new pattern entry is created. Over time, Layer 4's coverage grows. This is the mechanism by which the system transcends its initial human ontology — slowly, through accumulated operational experience, not through a single architectural move.

Obligation Merge Pipeline

After each verified step, all three layers run in parallel. Their outputs merge into a single obligation set.

Merge Rules:

1. **Deduplication:** Obligations from different layers describing the same search space are merged. The higher-priority layer's metadata wins.
2. **Supersession:** If Layer 4 generates obligations for a constraint, Layer 1 self-tags for the same constraint are discarded. Layer 4's enumeration is more reliable.
3. **Augmentation:** Layer 3 obligations covering search spaces Layer 4 cannot compute (strategic, structural) are added without conflict.
4. **Absence rule:** If Layer 1 says "no obligations" but Layers 3 or 4 say "obligations exist," the obligations exist. Layer 1 is never trusted to assert absence.
5. **Confidence scoring:** Layer 4: 0.95 (mechanical, for recognized patterns). Layer 3: 0.7 initial (model-based, calibrates over time). Layer 1: 0.4 (hint channel).
6. **v2.2 Precision governor:** Obligations from any layer with confidence below 0.3 (after calibration) are held in a `tentative` state rather than immediately opened. They become full obligations only if corroborated by another layer or by the librarian's coverage analysis. This prevents low-confidence classifiers from flooding the table.

```
        async fn parse_obligations(step: &VerifiedStep, chain: &[Step],
problem: &Problem)
            -> Vec<Obligation>
        {
            // All three layers run in parallel
            let (self_tagged, classified, validator_obs) = tokio::join!(
                parse_solver_obligation_json(&step.raw_output),
```

```
                run_classifier_ensemble(problem, chain, step),
                sympy_obligation_extractor(&step.content,
&step.validator_result)
            );

            // Merge with priority: Layer 4 > Layer 3 > Layer 1
            // Apply precision governor to low-confidence results
            merge_obligations(self_tagged, classified, validator_obs)
        }
```

```
        CREATE TABLE obligation_sources (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          obligation_id TEXT NOT NULL REFERENCES obligations(id),
          source_layer INTEGER NOT NULL,
          source_detail TEXT,
          raw_output TEXT,
          confidence REAL NOT NULL,
          created_at TEXT NOT NULL
        );
```

Parallel Execution: Layer 1 is synchronous (parsed from solver output). Layer 3 is an async model call (~500ms). Layer 4 is a SymPy computation (~100ms). Total latency ≈ 500ms, concurrent with the solver's next step generation.

P3 Walkthrough: What Each Layer Catches

Step	Content	Layer 1	Layer 3	Layer 4
1	f(a) | a^a - f(a)^f(a)	Tags: substitution	No obligations	No obligations
3	f(1) = 1	Tags: conclusion	No obligations	No obligations
9	f(n) | n^n	Tags: lemma, **no branches**	**CATCHES:** search space open across families	**CATCHES:** enumerates divisor families for n=2,3,4,5
10	f(p) = p^k for prime p	Tags: case analysis	**CATCHES:** composites unexplored	Confirms: power family obligations open
17	max ratio is 3	Tags: bound claim	**CATCHES:** needs adversarial check	**CATCHES:** opens adversarial obligation for ratio > 3

Step 9 is the critical moment. Layer 1 misses it entirely. Layers 3 and 4 together generate the obligations that prevent premature convergence on c = 3.

Part 5: Obligation Lifecycle and Management

Obligation Schema

```
        CREATE TABLE obligations (
          id TEXT PRIMARY KEY,
          attempt_id TEXT NOT NULL,
          branch_id INTEGER DEFAULT 0,
          parent_node_id TEXT NOT NULL,
          description TEXT NOT NULL,
          obligation_type TEXT NOT NULL,
          priority REAL DEFAULT 0.5,
          confidence REAL DEFAULT 0.7,
          source_layer INTEGER,
          status TEXT DEFAULT 'open',
          assigned_model TEXT,
          closure_node_id TEXT,
          closure_type TEXT,
          escalation_level INTEGER DEFAULT 0,
          steps_spent INTEGER DEFAULT 0,
          max_steps INTEGER DEFAULT 20,
          budget_multiplier REAL DEFAULT 1.0,
          parallel_assignments INTEGER DEFAULT 1,
          search_space TEXT,
          superseded_by TEXT,         -- v2.2: references obligation or
node that invalidated this
          retraction_reason TEXT,     -- v2.2: why this was retracted
(false positive)
          created_at TEXT NOT NULL,
          closed_at TEXT
        );
```

Lifecycle States

```
open → assigned → closed (proved | refuted)
                → superseded (general result invalidated this)
                → retracted (false positive, shouldn't have been
opened)
                → demoted (budget exhausted after full
escalation)
```

Open: Created by obligation parsing pipeline. Waiting for orchestrator assignment.

Assigned: Orchestrator has assigned a model. Steps being generated.

Closed (proved): A closure node resolves this obligation. The search space was explored and resolved.

Closed (refuted): A counterexample was found. May invalidate parent claims.

Superseded (v2.2): A general result makes this obligation irrelevant. Example: if Step 15 proves "all multiplicative functions have ratio ≤ 2," then individual obligations to check specific multiplicative functions are superseded. Mass supersession is triggered when a verified node's mathematical scope contains the search space of open obligations.

Retracted (v2.2): The obligation was a false positive — it described a search space that doesn't exist for this problem. Retraction is triggered when: (a) the assigned model demonstrates in ≤ 3 steps that the obligation is vacuous, or (b) a validator computation shows the search space is empty, or (c) the rolling critic flags it as spurious. Retractions feed back to Layer 3 calibration as negative signal.

Demoted: Step budget exhausted after full escalation ladder (see below). Logged as gap. Reduces confidence but does not block completion.

The Completeness Invariant

```
while open_obligations(attempt_id) > 0 && total_budget > 0:
    assign_next_obligation()
```

The proof cannot terminate with open obligations. This replaces text-matching heuristics. A proof is complete when: all obligations are closed, superseded, retracted, or demoted, AND a conclusion node has been verified.

Honest caveat: This invariant enforces coverage of *detected* obligations. Obligations that were never detected (unknown unknowns) are invisible to it. The invariant provides resource-bounded epistemic closure, not mathematical completeness.

Obligation Explosion Prevention (v2.2)

The union semantics of the merge pipeline, combined with continuous monitoring agents, create a structural bias toward obligation generation. Without controls, obligations can grow faster than they close. This section specifies the controls.

Fan-out limits: A single proof step can generate at most `MAX_OBLIGATIONS_PER_STEP` obligations (default: 20). If the merge pipeline produces more, they are ranked by priority and the lowest are held in `tentative` status. Tentative obligations promote to `open` only if budget is available after higher-priority obligations are addressed.

Global ceiling: An attempt can have at most `MAX_OPEN_OBLIGATIONS` simultaneously open (default: 100). New obligations beyond this ceiling enter `tentative` status. The ceiling is a pressure valve, not a hard limit — tentative obligations promote as active ones close.

Mass supersession: When a verified node proves a general result, the orchestrator runs a supersession check: for each open obligation, does the new result's mathematical scope contain the obligation's search space? This is a lightweight model call (Haiku-class) that compares the general result against each open obligation. Obligations that are subsumed are moved to `superseded` status in batch.

```
        CREATE TABLE supersession_events (
          id TEXT PRIMARY KEY,
          attempt_id TEXT NOT NULL,
          triggering_node_id TEXT NOT NULL,
          obligations_superseded TEXT NOT NULL,  -- JSON array of
obligation IDs
          reasoning TEXT NOT NULL,
          created_at TEXT NOT NULL
        );
```

Retraction mechanism: When an assigned model determines within 3 steps that an obligation is vacuous (the described search space doesn't exist), it returns a retraction recommendation. The orchestrator validates this against the obligation's source layer and, if confirmed, moves the obligation to `retracted` status. This feeds back as negative signal to the classifier that generated it.

Tentative obligations schema:

```
        CREATE TABLE tentative_obligations (
          id TEXT PRIMARY KEY,
          attempt_id TEXT NOT NULL,
          parent_node_id TEXT NOT NULL,
          description TEXT NOT NULL,
          obligation_type TEXT NOT NULL,
          priority REAL,
          source_layer INTEGER,
          source_confidence REAL,
          reason_held TEXT,             -- 'fan_out_limit',
'global_ceiling', 'low_confidence'
          promoted_to TEXT,             -- obligation ID if promoted
          discarded_reason TEXT,        -- why it was never promoted
          created_at TEXT NOT NULL
        );
```

Escalation Ladder (Replaces Immediate Demotion)

When an obligation exhausts its step budget, it does NOT immediately demote. Instead:

1. **Reassign** to different model at higher temperature. Different blind spots.
2. **Decompose:** LLM breaks the obligation into smaller sub-obligations.

3. **Escalate:** Assign to a more capable model with higher thinking budget.
4. **Research:** Agent searches arxiv/mathoverflow for the specific obstacle.
5. **Council consult:** Bring the stuck obligation to council for strategic guidance.
6. **Demote:** Only after all escalation steps fail.

```
-- escalation_level in obligations table:
-- 0: initial assignment
-- 1: reassigned to different model
-- 2: decomposed into sub-obligations
-- 3: escalated to higher-capability model
-- 4: research pass (arxiv/mathoverflow)
-- 5: council consulted
-- 6: demoted (all escalation exhausted)
```

Part 6: Proof DAG Architecture

The architecture replaces the linear step chain with a directed acyclic graph of typed nodes.

Node Type System

Node Type	Description	Obligations	Validator
claim	Mathematical assertion	None (leaf)	SymPy / Lean
subgoal	Decomposition	Opens one per sub-part	Structural
reduction	WLOG, sufficiency	Opens for reduced form	SymPy equivalence
case_split	Exhaustive cases	Opens one per case	Completeness check
construction	Example/ counterexample	Closes parent on verify	SymPy evaluation
bound	Upper/lower bound	Opens adversarial	SymPy + adversary
obligation	Open task	Must be closed or demoted	Lifecycle tracking
closure	Resolution	Closes referenced obligation	Content-dependent
conflict	Two paths contradict	Opens review	Structural detection
audit	Exploration breadth	May open new obligations	Meta-reasoning
adversarial	Counterexample attempt	Closes or strengthens parent	SymPy evaluation
working_note	Agent observation	None (intelligence artifact)	Not validated

Node Schema

```
        CREATE TABLE proof_nodes (
          id TEXT PRIMARY KEY,
          attempt_id TEXT NOT NULL REFERENCES attempts(id),
          branch_id INTEGER DEFAULT 0,
          node_type TEXT NOT NULL,
          parent_ids TEXT,
          content TEXT NOT NULL,
          formal_content TEXT,
          technique_class TEXT,
          construction_family TEXT,
          status TEXT DEFAULT 'proposed',
          validator_used TEXT,
          validator_result TEXT,
          model_id TEXT,
          obligation_ref TEXT,
          opens_obligations TEXT,
          parse_layer_source TEXT,
          token_cost INTEGER,
          sequence_number INTEGER NOT NULL,   -- v2.2: monotonic for
ordering
          created_at TEXT NOT NULL,
          verified_at TEXT
        );
```

Concurrency Model (v2.2)

With 8-12 agents writing to the same DAG concurrently, we need explicit concurrency semantics.

Event-sourced DAG: All mutations to the proof DAG are expressed as immutable events appended to an event log. The DAG state is a projection of the event log. This eliminates race conditions on the DAG itself — agents append events, and the projection is rebuilt deterministically.

```
        CREATE TABLE dag_events (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          attempt_id TEXT NOT NULL,
          event_type TEXT NOT NULL,      -- 'node_proposed',
'node_verified', 'node_rejected',
                                         -- 'obligation_opened',
'obligation_closed', etc.
          payload TEXT NOT NULL,         -- JSON: the event data
          agent_role TEXT NOT NULL,
          sequence_number INTEGER NOT NULL,  -- monotonic within attempt
          created_at TEXT NOT NULL
        );
```

Obligation assignment uses optimistic locking: When the orchestrator assigns an obligation, it checks that the obligation's status hasn't changed since it was read. If another agent closed or superseded it in the meantime, the assignment is retried with the next-highest priority obligation.

Working note visibility: Agents see working notes as of their last DAG query, not as of the current instant. This is acceptable because working notes are advisory, not authoritative. Slight staleness is preferable to complex synchronization.

Part 7: Multi-Model Division of Labor

Role Assignments

Role	Assignment	Produces	Mode
Workers (1-3)	Frontier reasoning models	claim, construction, closure	Assigned obligations + 15% speculative
Formalizer (4)	Lean-capable model	claim (formal)	Incremental on chains ≥ 3
Adversary (5)	Strong reasoning model	adversarial, conflict	Speculative + attack
Librarian (6)	Fast retrieval model	audit, obligation	Continuous coverage monitoring
Rolling Critic	Haiku-class model	working_note	Continuous DAG review

Worker Speculation Budget

Each worker gets 15-20% of its token allocation for self-directed exploration after addressing assigned obligations. Speculative findings become working notes. If a speculation produces a verified claim with a novel technique, it auto-generates a new obligation. Workers can discover obligations the parsing pipeline missed.

The Adversary

The adversary operates in two modes simultaneously:

Speculative mode (from first constraint): The moment the solver establishes a structural constraint, the adversary begins independently searching for extremal constructions. It doesn't know what bound the solver will claim — it's trying to find the LARGEST possible ratio, period. This runs as a low-priority background process.

Attack mode (on bound claims): Triggered by verified bound nodes. High priority, focused refutation. Uses speculative findings as a head start.

v2.2: Universal attack scope. The adversary doesn't just attack bounds. It attacks:

- Bound claims: Find constructions exceeding the bound.
- Exhaustiveness claims: "We only need to check primes" — try composite inputs.
- Uniqueness claims: "f(1) = 1 is the only possibility" — try alternatives.
- Sufficiency claims: "It suffices to show X" — construct scenario where X holds but conclusion fails.
- Construction claims: "f(n) = n is bonza" — search for violating inputs.
- Case split completeness: "Cases: p = 2, p odd" — check for missed cases.

Budget scales with claim importance, not claim type.

v2.2: Coverage-Modulated Confidence.

When the adversary exhausts its budget without finding a counterexample, the system does NOT simply increase confidence in the bound. Instead, confidence adjustment is modulated by the adversary's *coverage* of the construction space:

```
        coverage_ratio = (
            adversary_families_explored / total_known_families
        )

        confidence_boost = base_boost * coverage_ratio

        # If adversary only searched 2 of 5 known families,
        # confidence boost is 40% of maximum.
        # If adversary searched 5 of 5, full boost.
        # If there are likely unknown families (Layer 4 flagged
        # unrecognized structure), coverage_ratio is capped at 0.7.
```

This prevents the dangerous failure mode where adversary budget exhaustion is misinterpreted as evidence for the bound when the adversary simply searched too narrow a space.

Dynamic adversary budget: The orchestrator monitors adversary progress via working notes:

- Promising lead (values approaching bound): increase budget up to 3x.
- Diminishing returns (same results repeatedly): decrease and demote early.
- Breakthrough (counterexample found): budget irrelevant, emit conflict node.
- Research needed: pause, trigger arxiv search, resume with results.

The Librarian: Continuous Coverage Monitor

The librarian runs on every verified node as a lightweight ping: `{node_type, technique_class, construction_family}`. It maintains a running coverage map against the pre-solve briefing's technique ranking.

Trigger conditions for full audit: Coverage drops below threshold, OR solver has spent 3+ consecutive steps in the same technique family, OR it's been 10+ steps since last audit.

Anti-context retrieval: When the DAG is dense with one technique, the librarian retrieves the *least similar* successful technique for the problem class and injects it as a working note.

Active research: When the librarian detects a coverage gap, it proactively searches arxiv and mathoverflow via MCP. Findings are injected as working notes with obligation suggestions.

```
CREATE TABLE coverage_snapshots (
  id TEXT PRIMARY KEY,
  attempt_id TEXT NOT NULL,
  step_number INTEGER NOT NULL,
  techniques_explored TEXT NOT NULL,
  techniques_missing TEXT NOT NULL,
  coverage_score REAL NOT NULL,
  tunnel_depth INTEGER DEFAULT 0,
  alert_fired BOOLEAN DEFAULT false,
  created_at TEXT NOT NULL
);
```

The Formalizer: Incremental, Not Deferred

The formalizer begins formalizing any verified chain of ≥3 consecutive claims as they accumulate. Benefits:

- **Early gap detection:** A claim that can't be formalized in Lean is a red flag while the proof is still running.
- **Incremental output:** By completion, 60-80% is already formalized.
- **Formalization feedback loop:** Failed attempts generate specific feedback ("cannot unify types") that returns to the solver as a working note.

Rolling Critic

A Haiku-class critic monitors the DAG continuously. Fires every 10 verified nodes, on anomalies, on adversary working notes, when coverage drops below 0.4, or when the solver has spent 5+ consecutive steps in the same technique family. Produces a 1-paragraph assessment as a working note. Cost: ~$0.002 per firing.

Continuous Council (v2.2)

Three checkpoints rather than post-mortem only:

Early council (30% of step budget): "Is the solver on a productive path? Should we branch? What hasn't been tried?" On P3, this would have caught the exclusive focus on power functions by step 6.

Mid council (60% of step budget): "Are we converging? Is the adversary finding counterexamples? Should we reallocate?" Strategic rebalancing.

Final council (on completion): Full post-mortem with critic, cartographer, and synthesizer.

Cost: ~$0.50 per checkpoint. Three checkpoints: ~$1.50.

Part 8: Context Management (v2.2)

The Problem

With mandatory working notes, continuous librarian pings, adversary speculations, and rolling critic observations, the DAG accumulates substantial meta-commentary. When agents query the DAG, they risk drowning in noise rather than finding mathematical content.

Context Window Management

Agents access the DAG through context tools that enforce relevance and token budgets:

dag_query returns nodes ranked by relevance to the querying agent's current obligation, with a configurable token budget. Default: 4000 tokens. Mathematical content nodes rank higher than working notes. Recent nodes rank higher than old nodes. Nodes in the same technique family as the current obligation rank highest.

trace_chain returns the verification chain to a specific node. No filtering — chains are always relevant.

get_failures returns rejected nodes filtered by technique/construction. Working notes are excluded unless they contain failure analysis.

obligation_query returns obligations matching criteria. Compact format (description + status + priority), expandable on request.

```
        # Agent context tool with relevance filtering
        def dag_query(
            node_types: list[str],
            technique_class: str = None,
            status: str = None,
```

```
            token_budget: int = 4000,
            relevance_to: str = None,    # obligation ID for relevance
scoring
            exclude_note_types: list[str] = None  # e.g.,
['librarian_ping', 'critic_routine']
        ) -> list[Node]:
            # Fetch matching nodes
            # Score by relevance to current obligation
            # Truncate to token budget
            # Return highest-relevance nodes first
            ...
```

Working note classification: Working notes are tagged by type: `adversary_observation`, `technique_hint`, `failure_analysis`, `librarian_ping`, `critic_routine`, `convergence_check`. Agents can filter by type. Routine pings (librarian, critic) are excluded from default queries; only substantive observations surface.

Context Append: Mandatory Working Notes

Every agent produces a structured working note after every action. This is parsed from the agent's output, not an additional call.

```
        {
          "agent_role": "adversary",
          "action": "attempted counterexample",
          "obligation_id": "obl_17_adversarial",
          "result": "f(n) = 2^(v_2(n)) achieves ratio 4 at n=4",
          "insight": "Exponential constructions via p-adic valuation
bypass power function ceiling",
          "note_type": "adversary_observation",
          "substantive": true,
          "next_recommendation": "Solver should verify bonza condition
for this construction",
          "confidence": 0.9
        }
```

The `substantive` flag distinguishes notes that other agents should see from routine status updates. Non-substantive notes are stored but excluded from default dag_query results.

Part 9: OODA Self-Modification with In-Run Safety (v2.2)

Capability

When agents encounter runtime failures (validator bugs, parser errors, schema gaps), they can use OODA filesystem and shell access to fix the issue and continue.

Safety Model

The problem v2.1 identified: Self-modification is powerful and dangerous. Agents generating proofs can mutate the environment validating those proofs.

The problem v2.2 adds: Audit logs are forensics, not prevention. A subtle validator relaxation introduced to "fix an edge case" can become a silent soundness leak. The proof still "verifies." Obligations still close. Confidence still rises. No invariant detects validator drift.

In-run regression testing (v2.2): Any OODA modification to validation code triggers an immediate regression check before the proof run continues.

```
CREATE TABLE self_modifications (
  id TEXT PRIMARY KEY,
  agent_role TEXT NOT NULL,
  attempt_id TEXT,
  file_modified TEXT NOT NULL,
  modification TEXT NOT NULL,
  diff TEXT NOT NULL,
  reason TEXT NOT NULL,
  trigger_event TEXT,
  -- v2.2 additions:
  affects_validation BOOLEAN DEFAULT false,
  regression_suite_run BOOLEAN DEFAULT false,
  regression_passed BOOLEAN,
  regression_failures TEXT,       -- JSON: which tests failed
  reverted BOOLEAN DEFAULT false,
  created_at TEXT NOT NULL
);
```

Rules:

1. **Classification:** Every modification is classified as `affects_validation` or not. Modifications to validators, parsers, the obligation merge pipeline, or the completeness invariant check are `affects_validation = true`.
2. **Validation-affecting modifications** trigger an immediate regression suite: a set of known-good/known-bad mathematical expressions that

the validator must correctly accept/reject. The proof run PAUSES until the regression suite passes.
3. **If regression fails:** The modification is automatically reverted. The agent receives the regression failure as context and can attempt a different fix.
4. **The completeness invariant itself** is never modifiable by agents. This is enforced by filesystem permissions (the invariant check is in a read-only module).
5. **Non-validation modifications** (prompt templates, parsers for non-critical metadata, logging) proceed without regression testing but are still logged.
6. **Scope:** Agents may only modify files in a whitelist of non-critical paths. Validator core, DAG engine, and obligation lifecycle are read-only.

Proactive Health Monitoring

Between proof runs, a Health Monitor agent runs background checks:

- Validator edge case sweep against known failures
- MCP server health check
- Schema integrity validation
- Prompt template smoke test
- Dependency check (SymPy, Lean, API keys)

If any check fails, the next proof run doesn't start until resolved.

```
CREATE TABLE health_checks (
  id TEXT PRIMARY KEY,
  check_type TEXT NOT NULL,
  status TEXT NOT NULL,
  details TEXT,
  self_repaired BOOLEAN DEFAULT false,
  repair_mod_id TEXT,
  created_at TEXT NOT NULL
);
```

Part 10: Completion Quality Metric (v2.2)

Every proof conclusion carries a completion quality score that distinguishes genuine resolution from budget-shaped closure.

```
ALTER TABLE attempts ADD COLUMN completion_quality TEXT;
-- JSON object with the following fields:
-- {
--   "total_obligations": 47,
--   "closed_proved": 38,
--   "closed_refuted": 2,
--   "superseded": 4,
--   "retracted": 1,
```

```
        --   "demoted": 2,
        --   "genuine_closure_rate": 0.936,    -- (proved + refuted +
superseded) / total
        --   "demotion_rate": 0.043,
        --   "layer4_obligation_rate": 0.65,   -- fraction sourced from
Layer 4
        --   "adversary_coverage": 0.80,       -- fraction of known
families adversary searched
        --   "confidence_class": "high"        -- high / medium / low /
unreliable
        -- }
```

Confidence classes:

- **High:** genuine_closure_rate > 0.90, adversary_coverage > 0.70, demotion_rate < 0.05
- **Medium:** genuine_closure_rate > 0.75, adversary_coverage > 0.50
- **Low:** genuine_closure_rate > 0.50 OR demotion_rate > 0.20
- **Unreliable:** genuine_closure_rate < 0.50 OR adversary_coverage < 0.30

The confidence class is surfaced in the proof conclusion alongside the answer. A “low” or “unreliable” result is flagged for human review and does not propagate as institutional knowledge.

Part 11: Death Spiral Prevention (v2.2)

Detection

A lightweight monitor watches solver output for signals of prior-evidence conflict:

- **Hedging language:** “wait,” “actually,” “no,” “let me reconsider”
- **Numeric oscillation:** alternating between different values across steps
- **Self-contradiction:** step N claims X, step N+1 claims NOT X
- **Conclusion anchoring:** solver states “therefore c = K” but K doesn’t match the verified chain

Thresholds (Avoiding False Positives)

Single instances of hedging language are normal mathematical self-correction. The monitor fires only on:

- **Oscillation:** Same value pair appears 2+ times (e.g., “c = 3/2… no, c = 3… wait, c = 3/2”)
- **Contradiction:** Explicit negation of a verified claim within 3 steps
- **Anchoring:** Final answer differs from the value derived in the verified chain by more than a trivial rearrangement

Intervention

When fired: 1. Pause the solver 2. Surface the conflict to the orchestrator 3. Open a council review on the specific conflict 4. Fork the proof: one branch continues, another starts from the divergence point with explicit instruction to explore the alternative

```
        CREATE TABLE spiral_detections (
          id TEXT PRIMARY KEY,
          attempt_id TEXT NOT NULL,
          step_number INTEGER NOT NULL,
          detection_type TEXT NOT NULL,
          evidence TEXT NOT NULL,
          intervention TEXT NOT NULL,
          created_at TEXT NOT NULL
        );
```

Part 12: Cold Start and Bootstrapping (v2.2)

The Problem

The technique registry starts empty. The model affinity table starts empty. The layer calibration table starts empty. The pre-solve intelligence gathering depends on all three. Without bootstrapping, the system's initial capability is exactly as good as the human operator's mathematical ontology.

Bootstrap Protocol

Phase 1: Manual seeding. Populate the technique registry with standard technique families for competition mathematics. This is a one-time human investment of ~2-4 hours, producing ~50-100 registry entries across 5-10 problem classes. A seed file ships with ChatDB.

```
        -- Example seed entries
        INSERT INTO technique_registry (problem_class, technique_family,
description, source)
        VALUES
          ('functional_equation', 'substitution', 'Substitute specific
values for variables', 'seed'),
          ('functional_equation', 'power_function', 'Try f(n) = n^k
family', 'seed'),
          ('functional_equation', 'exponential', 'Try f(n) = a^(v_p(n))
constructions', 'seed'),
          ('functional_equation', 'multiplicative',
'Try f(mn) = f(m)f(n) constructions', 'seed'),
          ('number_theory', 'modular_arithmetic', 'Work modulo small
primes', 'seed'),
          ...
```

Phase 2: Calibration runs. Run 10-20 problems with known answers across diverse domains. These runs populate model affinity data, layer calibration data, and expand the technique registry with extracted patterns. This is the "test set" that validates the architecture beyond P3.

Phase 3: Operational learning. Every subsequent run updates the registry, calibration, and affinity tables. The system improves with use.

Cross-Attempt Intelligence

When a new attempt starts for the same problem:

1. Load all prior attempts' council findings
2. Load demoted obligations (things that need different approaches)
3. Load adversarial counterexamples
4. Pre-seed obligation table with unresolved obligations from prior attempts
5. Inject death spiral detections as warnings

```
        CREATE TABLE cross_attempt_context (
          id TEXT PRIMARY KEY,
          problem_id TEXT NOT NULL,
          source_attempt TEXT NOT NULL,
          context_type TEXT NOT NULL,
          content TEXT NOT NULL,
          priority REAL DEFAULT 0.5,
          consumed_by TEXT,
          created_at TEXT NOT NULL
        );
```

Part 13: Technique Registry

Schema

```
        CREATE TABLE technique_registry (
          id INTEGER PRIMARY KEY,
          problem_class TEXT NOT NULL,
          technique_family TEXT NOT NULL,
          description TEXT,
          success_count INTEGER DEFAULT 0,
          failure_count INTEGER DEFAULT 0,
          avg_steps_to_close REAL,
          exemplar_construction TEXT,
-- the actual construction that succeeded
          key_insight TEXT,                -- the reasoning that led to
success
          failure_modes TEXT,              -- JSON: known failure modes
          prerequisite_techniques TEXT,    -- JSON: technique families
```

```
that typically precede
          example_problem TEXT,
          example_node_id TEXT,
          source TEXT DEFAULT 'extracted',
          created_at TEXT NOT NULL,
          last_used_at TEXT
        );
```

Real-Time Updates

The registry updates DURING proof runs, not just after:

- Worker closes obligation with novel technique → insert immediately. Other workers on the same run benefit.
- Adversary discovers new construction family → insert immediately.
- Librarian's arxiv search finds technique → insert with source='arxiv_search'.
- Classifier detects technique not in registry → flag for insertion.

Post-attempt council review performs deep curation (merging duplicates, updating rates, adding descriptions). But the registry is a live document.

Batch-Level Learning

Registry updates from problem N propagate immediately to problem N+1 within a batch. Institutional learning compounds within a batch, not just across sessions.

Institutional Learning Metrics

- **Steps to first verified lemma:** Should decrease on known problem classes.
- **Completeness deficit:** Open obligations at proof completion. Should trend toward zero.
- **Adversarial survivability:** Percentage of bound claims surviving adversary attack.
- **Obligation closure efficiency:** Obligations closed per token.
- **Parse layer agreement:** How often Layers 1, 3, 4 agree.
- **Pre-solve hit rate:** How often briefing predicts eventual solution technique.
- **Speculative adversary lead time:** Steps before bound claim does adversary find counterexample.
- **Reassignment success rate:** How often 2nd/3rd attempt closes what 1st couldn't.

Part 14: External Control Surface

ChatDB is a standalone Tauri application exposing an HTTP API and webhook system.

API Endpoints

```
POST /api/problems              Create a problem
GET  /api/problems/:id          Get problem + state
POST /api/problems/:id/solve    Start/continue proof run
POST /api/problems/:id/pause    Pause
POST /api/problems/:id/stop     Stop and finalize
GET  /api/attempts/:id          Get attempt state
GET  /api/attempts/:id/dag      Get full proof DAG
GET  /api/attempts/:id/obligations  Get obligations
POST /api/attempts/:id/inject   Inject node or obligation
externally
GET  /api/attempts/:id/aar      Get After Action Report
GET  /api/attempts/:id/quality  Get completion quality metric
(v2.2)
POST /api/attempts/:id/council  Trigger council review
GET  /api/registry              Query technique registry
POST /api/registry              Add technique entry
GET  /api/models                List model profiles
PUT  /api/models/:id            Update model config
```

Webhooks

```
proof.step.verified      { node_id, content, validator,
obligations_opened }
proof.step.rejected      { node_id, content, reason }
proof.obligation.opened  { obligation_id, type, source_layer,
priority }
proof.obligation.closed  { obligation_id, closure_type,
closure_node_id }
proof.obligation.superseded  { obligation_id, superseding_node,
count }  -- v2.2
proof.obligation.retracted   { obligation_id,
reason }                   -- v2.2
proof.obligation.demoted { obligation_id, steps_spent, reason }
proof.branch.created     { branch_id, fork_reason }
proof.branch.conflict    { branch_a, branch_b, conclusions }
proof.bound.claimed      { node_id, bound_value }
proof.adversary.result   { node_id, counterexample_found,
coverage_ratio }  -- v2.2
proof.conclusion         { answer, confidence,
completion_quality }  -- v2.2
proof.council.triggered  { reason, findings }
proof.death_spiral       { step_number, conflicting_values }
proof.self_modification  { file, reason, affects_validation,
regression_passed }  -- v2.2
```

External MCP Server Wrapper

A thin MCP server (separate project) wraps the ChatDB API so any MCP client can drive proof runs:

```
chatdb_create_problem     → POST /api/problems
chatdb_start_solve        → POST /api/problems/:id/solve
chatdb_get_obligations    → GET  /api/attempts/:id/obligations
chatdb_inject_node        → POST /api/attempts/:id/inject
chatdb_get_dag            → GET  /api/attempts/:id/dag
chatdb_get_quality        → GET  /api/attempts/:id/quality
chatdb_trigger_council    → POST /api/attempts/:id/council
chatdb_query_registry     → GET  /api/registry
chatdb_configure_model    → PUT  /api/models/:id
```

Part 15: Model Configuration

```sql
CREATE TABLE model_configs (
  id TEXT PRIMARY KEY,
  role TEXT NOT NULL,
  model_id TEXT NOT NULL,
  temperature REAL DEFAULT 0.3,
  thinking_enabled BOOLEAN DEFAULT true,
  thinking_budget INTEGER DEFAULT 10000,
  max_output_tokens INTEGER DEFAULT 4096,
  system_prompt TEXT,
  active BOOLEAN DEFAULT true,
  notes TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT
);
```

Recommended Defaults

Role	Model	Temp	Thinking	Budget	Rationale
Solver (primary)	Claude Sonnet 4.6	0.3	Yes, 10k	4096	Conservative, deep reasoning
Solver (divergent)	DeepSeek R1	0.7	Yes, 16k	4096	Creative constructions
Adversary (speculative)	Claude Sonnet 4.6	0.6	Yes, 10k	4096	Creative extremal search
Adversary (attack)	Claude Sonnet 4.6	0.4	Yes, 10k	4096	Focused refutation
Classifier (L3-A)	Claude Haiku 4.5	0.0	No	1024	Deterministic pattern detection
Classifier (L3-B)	GPT-4o-mini	0.0	N/A	1024	Different blind spots
		0.0	No	1024	

Role	Model	Temp	Thinking	Budget	Rationale
Classifier (L3-C)	DeepSeek V3				Third perspective
Librarian	Claude Haiku 4.5	0.2	No	2048	Continuous monitoring
Formalizer	Claude Sonnet 4.6	0.0	Yes, 10k	8192	Incremental Lean output
Rolling Critic	Claude Haiku 4.5	0.1	No	1024	Cheap, frequent
Pre-solve (×4)	Claude Haiku 4.5	0.1-0.2	No	2048	Parallel intelligence gathering
Council (×3)	Mixed families	0.2-0.3	Yes	4096	Blind spot diversity
Health Monitor	Claude Haiku 4.5	0.0	No	2048	Proactive system health

Key Principles:

- Different models for different roles. Same blind spots = same failures.
- Temperature reflects the role. Low for precision. Higher for creativity.
- Everything tunable via API. No recompile.
- When in doubt, add a model call. Haiku costs $0.001.
- Parallelize by default. The merge pipeline handles conflicts.

Part 16: Dynamic Confidence Calibration

Layer confidence scores (Layer 4: 0.95, Layer 3: 0.7, Layer 1: 0.4) are initial values that update based on rolling accuracy via Bayesian updating.

```
CREATE TABLE layer_calibration (
  layer INTEGER NOT NULL,
  classifier_id TEXT,
  obligations_generated INTEGER DEFAULT 0,
  obligations_closed INTEGER DEFAULT 0,
  obligations_demoted INTEGER DEFAULT 0,
  obligations_retracted INTEGER DEFAULT 0,   -- v2.2
  obligations_useful INTEGER DEFAULT 0,
  calibrated_confidence REAL,
  updated_at TEXT
);
```

If Layer 3 Classifier B's obligations are retracted (false positives) more than 30% of the time, its confidence drops and its obligations enter `tentative` by default. If Layer 4 obligations close 95% of the time, its score stays high.

Prompt Evolution

After every N completed attempts (default N=5), a meta-agent reviews performance data and proposes prompt modifications. Changes propagate to the next run. Performance is tracked per prompt version.

```
CREATE TABLE prompt_versions (
  id TEXT PRIMARY KEY,
  role TEXT NOT NULL,
  version INTEGER NOT NULL,
  system_prompt TEXT NOT NULL,
  performance_metrics TEXT,
  promoted BOOLEAN DEFAULT false,
  created_at TEXT NOT NULL,
  created_by TEXT
);
```

Part 17: Implementation Roadmap

Realistic timeline: 6 months at 2-4pm daily build window.

Sprint 1: Obligation Parsing Pipeline (Weeks 1-4)

The foundation. Without reliable obligation parsing, the rest is inert. This is a research-grade integration problem (natural language → structured SymPy expressions).

1. Layer 1: JSON sidecar format in solver prompt, parse self-tags.
2. Layer 4: `obligation_extractor.py` with initial pattern library (divisibility, bound, modular, function space patterns). The NL→SymPy parsing is the hard part here.
3. Layer 3: Classifier prompt, single model first, expand to ensemble.
4. Merge pipeline with supersession, augmentation, contradiction handling, confidence scoring, precision governor.
5. `obligation_sources` table.
6. **Validation:** Run P3. At Step 9, Layer 4 generates ≥ 3 construction family obligations. Layer 3 flags search narrowing.

Sprint 2: Obligation Engine + Completeness Invariant (Weeks 5-8)

1. `proof_nodes` and `obligations` tables with full lifecycle (including `superseded`, `retracted`).
2. Event-sourced DAG with `dag_events` table.
3. Completeness invariant: loop cannot exit with open obligations.
4. Obligation explosion controls: fan-out limits, global ceiling, tentative queue.
5. Mass supersession mechanism.
6. Retraction mechanism with calibration feedback.

7. Orchestrator: priority scoring, model assignment, optimistic locking.
8. **Validation:** Run P3 with obligations. System cannot conclude c=3 while construction family obligations remain open.

Sprint 3: Multi-Model + Adversary (Weeks 9-14)

1. Adversary with speculative + attack modes, universal claim scope.
2. Coverage-modulated confidence on adversary survival.
3. Worker pool with obligation-based assignment + speculation budget.
4. Librarian with continuous coverage monitoring and anti-context retrieval.
5. Escalation ladder (6 levels before demotion).
6. Context management: relevance-filtered dag_query, token budgets, working note classification.
7. **Validation:** Adversary finds ratio > 3 counterexample. Completion quality metric reflects genuine vs. demoted closure.

Sprint 4: Registry + Learning + Safety (Weeks 15-18)

1. Technique registry with real-time updates, exemplar constructions, key insights.
2. OODA self-modification with in-run regression testing for validation-affecting changes.
3. Death spiral detection with calibrated thresholds.
4. Completion quality metric on every proof conclusion.
5. Cold start: seed file + calibration run protocol.
6. Cross-attempt intelligence.
7. **Validation:** Run calibration suite (10-20 diverse problems). Registry grows. Affinity data populates.

Sprint 5: API + Formalization + Polish (Weeks 19-24)

1. HTTP API server (Axum, embedded in Tauri).
2. Webhook emitter with event filtering.
3. Formalizer role with incremental Lean output.
4. Continuous council (early/mid/final checkpoints).
5. Rolling critic integration.
6. External MCP server wrapper (separate repo).
7. Full AAR with obligation coverage, parse layer stats, completion quality, adversarial results.
8. Known answer mode with conclusion mismatch detection.
9. **Validation:** External MCP wrapper drives full P3 run end-to-end. System produces c=4 with high completion quality.

Verification Criteria

Sprint	Test	Pass Condition
1	Parse P3 Step 9	Layer 4: ≥ 3 construction obligations. Layer 3: flags narrowing.
2		

Sprint	Test	Pass Condition
	Run P3 with engine	Cannot conclude c=3 with open obligations. Supersession works.
3	Multi-model P3	Adversary finds ratio > 3. Coverage-modulated confidence.
4	Calibration suite	10+ diverse problems. Registry grows. No regression from OODA mods.
5	End-to-end	c=4 via API. Completion quality: high. All systems integrated.

Part 18: Cost Projections

Tier	Architecture	Est. Cost/Problem	Confidence
M1 (current)	Linear step-verify, 1 model	$0.50-$1.00	Locally correct, globally incomplete
Sprint 1-2	Obligations + 3-layer parse + pre-solve	$3-$8	Mechanical completeness pressure
Sprint 3-4	Multi-model + adversary + librarian + safety	$15-$40	Adversarial completeness + self-monitoring
Sprint 5	Full system + formalization + API	$25-$60	Formally verified + continuously improving

Even at $60/problem, cost is 2-3 orders of magnitude below frontier lab approaches. Every monitoring call that catches one wrong answer per 100 runs pays for itself.

Part 19: Research Output

Paper 1: “Verification Is Not Exploration: Step-Level Proof Engines and the Completeness Gap.” The M1 result. Evidence: IMO P3 run (19/19 verified, wrong answer).

Paper 2: “Mechanical Completeness Enforcement via Typed Obligation Graphs with Adversarial Refutation.” The full architecture. Evidence: before/after on competition problems, institutional learning rate, completion quality metrics.

Paper 3: “Hybrid Obligation Parsing for Mathematical Proof Search: Combining Symbolic Computation with Model-Based Classification.” The parsing pipeline. Evidence: layer agreement rates, obligation detection recall, retraction rates, parse layer quality.

Part 20: Known Attack Surface

This section documents the known limitations that future versions should address. These are not bugs — they are honest boundaries of the current design.

A. Layer 4 domain scope. The pattern library is finite and grows slowly. Problems requiring construction families not in the library degrade to Layer 3 quality. Mitigation: the `obligation_patterns` registry, council-driven pattern extraction, and the meta-pattern for unrecognized structure. This is a growing library, not a complete enumeration.

B. Correlated classifier blind spots. If no model in the Layer 3 ensemble has training signal on a relevant technique, the union of three zeros is still zero. Mitigation: arxiv search in pre-solve and continuous librarian research. This is an irreducible limitation of model-based classification.

C. Adversary constructive-only search. The adversary finds concrete counterexamples. It cannot reason about asymptotic behavior or constructions that only dominate in the limit. Future work: an asymptotic reasoning mode that uses SymPy limit computation.

D. Resource-bounded closure. Different budgets produce different "complete" proofs. The completion quality metric makes this visible but doesn't eliminate it.

E. Human ontology dependency. The technique registry, pattern library, and obligation extractors encode human mathematical culture. The system works well where that culture is dense (competition math) and degrades where it's sparse (novel research). This is a feature, not a bug — but it bounds the system's ambition.

F. Context window pressure. On large DAGs (100+ nodes), agent context queries may not capture all relevant information. The relevance filtering helps but is heuristic. Future work: progressive summarization of DAG regions.

Appendix A: Full Schema Reference

```
        -- Core tables
        CREATE TABLE problems (...);
        CREATE TABLE attempts (...);
        CREATE TABLE proof_nodes (...);
        CREATE TABLE obligations (...);
        CREATE TABLE tentative_obligations (...);
        CREATE TABLE dag_events (...);

        -- Obligation parsing
        CREATE TABLE obligation_sources (...);
```

```
CREATE TABLE obligation_patterns (...);
CREATE TABLE supersession_events (...);

-- Intelligence
CREATE TABLE pre_solve_briefings (...);
CREATE TABLE technique_registry (...);
CREATE TABLE coverage_snapshots (...);
CREATE TABLE cross_attempt_context (...);

-- Model management
CREATE TABLE model_configs (...);
CREATE TABLE model_affinity (...);
CREATE TABLE layer_calibration (...);
CREATE TABLE prompt_versions (...);

-- Safety and monitoring
CREATE TABLE self_modifications (...);
CREATE TABLE health_checks (...);
CREATE TABLE spiral_detections (...);
CREATE TABLE node_annotations (...);

-- External
CREATE TABLE mcp_servers (...);
CREATE TABLE agent_mcp_access (...);
CREATE TABLE batch_runs (...);
CREATE TABLE background_tasks (...);
```
